

ACP_A2A_PROVIDER

HelixAgent ACP via Google A2A (Agent2Agent) — Design Spike (P4-T4)

Status	DESIGN (spike before implementation, §11.4.6 — do not code the A2A surface until this is agreed)
Scope	The ACP → Google A2A interop capability: HelixAgent exposes an A2A server (other agents/CLI tools reach HelixAgent, incl. the local HelixLLM) + an A2A client (HelixAgent calls out to other A2A agents)
Owns	Implementation-plan item P4-T4 (04_implementation_plan.md:93, :132)
Created	2026-07-07 · Revision 2 · Track (T1/main) · Branch feature/helixllm-full-extension
Revision 2 (2026-07-07)	All A2A protocol facts INDEPENDENTLY re-fetched + verified against the LATEST spec (a2a-protocol.org/latest/ + /v0.3.0/ + a2aproject/A2A GitHub, 2026-07-07); §2.1 corrected to source-verified evidence — the pre-existing proprietary /api/v1/acp/{execute,broadcast,status} routes are canned-response STUBS (cmd/api/main.go:211-215, :370-418), honestly distinguished from the Google A2A wire (§11.4.124); no-/v1 base-URL gotcha made explicit (§2.2); proto-vs-JSON dual-surface tightened (§1.1); Q6 (operator-gated stub fate, §11.4.122) added
Operator decision (binding)	

	03_open_clarifications.md C3 (2026-07-06): “ Google A2A (Agent-to-Agent) — implement A2A interop, NOT Zed ACP.” This document HONOURS that decision; it does NOT re-decide the protocol (§11.4.6 / §11.4.66).
Grounding	docs/research/ 07.2026/00_master/ 03_open_clarifications.md C3 · 04_implementation_plan.md P4-T4 · submodules/helix_llm/docs/ API_CONTRACT.md (TLS/auth/route posture) · submodules/ helix_agent/cmd/api/main.go (:140-144 acp-agent- communication template; :162 UnifiedProtocolManager; :211-215+ :370-418 the pre-existing proprietary /api/v1/ acp/{execute,broadcast,status} STUB routes/handlers) · submodules/ helix_agent/internal/router/ router.go (gin route surface) · submodules/helix_agent/ internal/mcp/ (MCP host/bridge, the agent↔ tool boundary) · LATEST A2A spec (§11.4.99 — fetched, cited below)

Anti-bluff (§11.4.6): every latency/throughput figure below is flagged (EST – measure) — no benchmark here is a captured measurement. Protocol facts are cited to the LATEST official A2A spec fetched 2026-07-07; nothing is taken from model memory (§11.4.99). Where the spec’s proto-first v1.0 surface and its JSON-RPC binding differ in naming, the tension is stated honestly (§0.1) rather than papered over.

0. Operator decision + why A2A over the other ACP candidates

0.1 What “ACP” resolved to (verbatim operator quote)

03_open_clarifications.md C3 recorded three live meanings of the acronym “ACP” and the operator’s resolution:

C3 “ACP” — research finding: “*Zed Agent Client Protocol (editor ↔ agent) vs IBM/BeeAI ACP vs Google A2A (agent ↔ agent).*” **OPERATOR RESOLUTION (2026-07-06):** “ Google A2A (Agent-to-Agent) — implement A2A interop, NOT Zed ACP. (Note: A2A is agent ↔ agent, so ‘editor auto-recognition’ is served separately via the /v1/models + MCP server/discover capability surface; A2A adds cross-agent interop.)”

The operator’s parenthetical is the load-bearing design constraint: **A2A is the agent ↔ agent lane; editor/tool auto-recognition is a DIFFERENT lane** already served by /v1/models (HelixCode) + MCP server/discover (Claude Code) + the capability surface (CONST-040). A2A does not replace those — it adds a peer-to-peer interop surface so HelixAgent can (a) be reached BY other A2A agents and (b) reach OUT to other A2A agents.

Why A2A over the two rejected candidates (recorded for transparency, not to re-decide): - **Zed ACP** (Agent Client Protocol) is an *editor ↔ agent* JSON-RPC-over-stdio protocol — it solves “an editor drives one local agent process”, which HelixCode already covers through the /v1/models + MCP surface. It is NOT an agent-to-agent interop standard. - **IBM / BeeAI ACP** is a competing agent-communication spec; the operator chose the Google/Linux-Foundation-governed A2A instead (broader multi-vendor backing, an official Go SDK — §1.6). - **Google A2A** is a vendor-neutral, Linux-Foundation-governed open standard for opaque-agent interoperability with a published Go SDK, whose transport reuses HTTP + JSON-RPC 2.0 + SSE — the exact primitives HelixAgent’s gin server already speaks (§2). That reuse (§11.4.74 extend-don’t-reimplement) is why it fits cleanly on the existing stack with no new transport.

1. What A2A is (grounded in the LATEST official spec)

A2A (Agent2Agent) is an open protocol for communication and interoperability between independent, potentially **opaque** AI-agent systems — an agent can expose and consume capabilities across vendor/framework boundaries without revealing its internal implementation, memory, or tools.

Sources (fetched 2026-07-07 via WebFetch; §11.4.99 latest-source): [A2A spec — latest](#) · [A2A spec — v0.3.0 \(JSON-RPC binding detail\)](#) · [a2aproject/A2A GitHub](#).

1.1 Governance + version (cited)

- **Governance:** the GitHub project page states verbatim “*The A2A Protocol is an open source project under the Linux Foundation, contributed by Google.*” (github.com/a2aproject/A2A, accessed 2026-07-07).

- **Latest released spec: v1.0.x** — the `a2aproject/A2A` releases page shows **v1.0.1 (released 2026-05-28)** as the latest, and the `/latest/specification/` page self-identifies its released version as **1.0.0** (accessed 2026-07-07). For this document, “the spec” = the v1.0 line; the JSON-RPC binding method-name strings in §2 are cross-checked against the v0.3.0 spec (which the v1.0 line is backward-compatible with on the JSON-RPC wire).

Honest version note (§11.4.6). The v1.0 spec is **proto-first**: it states *“the file `spec/a2a.proto` is the single authoritative normative definition of all protocol data objects and request/response messages”*, and its section headings use proto-style names — the `TaskState` enum members are `TASK_STATE_SUBMITTED` ... `TASK_STATE_UNSPECIFIED` (verified 2026-07-07 on `/latest/`), the `Message.role` enum is `ROLE_USER/ROLE_AGENT`, and the `Part` oneof is `text/raw/url/data`. The **JSON-RPC 2.0 binding** maps those onto the wire STRINGS shown in §2: method names (`"message/send"`), task-state values (`"submitted"`), and the proto `TASK_STATE_UNSPECIFIED` catch-all binds to JSON `"unknown"`, and `Part` kind values (`"text"`, `"file"`, `"data"` — the proto `raw/url` collapse into one JSON `FilePart` carrying `FileWithBytes` | `FileWithUri`). Both surfaces are the SAME protocol; §2 documents the JSON-RPC-over-HTTP(S) binding because that is the transport `HelixAgent` will implement (§3). This dual-surface is a spec fact I verified on both the `/latest/` (proto) and `/v0.3.0/` (JSON-RPC-binding) pages (2026-07-07), flagged so a reviewer can reconcile it — not an inconsistency in this design.

1.2 Agent Card (discovery)

An **Agent Card** is a public JSON document — an agent’s technical manifest — served (recommended) at the well-known URI `/.well-known/agent-card.json` (RFC 8615 well-known URI discovery). Cited fields (spec §4.4.1 `AgentCard` / v0.3.0 binding, accessed 2026-07-07):

- `name`, `description`, `version` — identity;
- `url` — the service endpoint an A2A client POSTs to;
- `capabilities` — boolean flags incl. `streaming`, `pushNotifications`, `extendedAgentCard`;
- `defaultInputModes` / `defaultOutputModes` — accepted media-type arrays;
- `skills` — an array of `AgentSkill` objects (`id`, `name`, `description`, `tags`, `examples`) advertising WHAT the agent can do;
- `securitySchemes` + `security` — declared auth (§1.7);
- `preferredTransport` / `interfaces` — which binding(s) the agent speaks.

Discovery is: a client GETs the well-known Agent Card, reads the skills + transport + security, then sends work to url.

1.3 Task lifecycle

A unit of work is a **Task** with a server-assigned `id` and a `status.state`. The TaskState JSON string values (cited, `v0.3.0` binding + `v1.0` TaskState enum, accessed 2026-07-07):

"submitted" → "working" → ("input-required" ↔ back to "working") → terminal one of "completed" / "canceled" / "failed" / "rejected" / "auth-required" ("unknown" is the catch-all). A Task carries a history of messages and produces artifacts (§1.4). Long-running tasks are first-class (the protocol is “async-first”).

1.4 Message / Part / Artifact types

- A **Message** has a `role` ("user" | "agent") and a `parts` array.
- A **Part** is discriminated by `kind` (cited JSON values, accessed 2026-07-07):
 - "text" — TextPart (a text string);
 - "file" — FilePart (bytes/base64 OR a url reference, + `mediaType`, `filename`);
 - "data" — DataPart (arbitrary structured JSON).
- An **Artifact** is a task OUTPUT composed of Part objects (the generated code, file, or structured result the client collects when the Task completes).

1.5 Transport — JSON-RPC 2.0 over HTTPS (+ gRPC, HTTP+JSON/REST)

The spec declares **three normative bindings** (`v1.0` §§9–11): **JSON-RPC 2.0 over HTTPS** (the most common), **gRPC**, and **HTTP+JSON/REST**. All requests/responses adhere to JSON-RPC 2.0. The JSON-RPC method-name STRINGS (cited verbatim from the `v0.3.0` binding, accessed 2026-07-07):

Method (JSON-RPC method)	Purpose
"message/send"	send a message → create/ continue a Task (non-streaming)
"message/stream"	send a message with an SSE stream of updates

Method (JSON-RPC method)	Purpose
"tasks/get"	fetch a Task's current state + artifacts
"tasks/cancel"	cancel an in-flight Task
"tasks/resubscribe"	re-attach an SSE stream to an existing Task
"tasks/pushNotificationConfig/set" //get //list //delete	manage webhook push-notification configs
"agent/getAuthenticatedExtendedCard"	fetch the authenticated (fuller) Agent Card

HelixAgent implements the **JSON-RPC 2.0 over HTTPS** binding (§3) because it maps 1:1 onto the existing gin + TLS server (§11.4.74 reuse); gRPC/REST bindings are a documented future extension, not shipped in P4-T4.

1.6 Official Go SDK

The GitHub project lists an **official Go SDK**: “ *A2A Go SDK* (github.com/a2aproject/a2a-go) — go get `github.com/a2aproject/a2a-go`” (github.com/a2aproject/A2A, accessed 2026-07-07). P4-T4 SHOULD consume this SDK for the wire types + JSON-RPC dispatch rather than hand-rolling the envelope (§11.4.74) — provided a build-time check confirms it targets Go 1.26 and the v1.0 wire (verify at implementation, flagged UNCONFIRMED until pinned).

1.7 Authentication

Auth is **declared in the Agent Card** (`securitySchemes + security`) and carried in transport headers — the protocol itself is credential-agnostic. Cited scheme types (v1.0 §4.5, accessed 2026-07-07): `APIKeySecurityScheme`, `HTTPAuthSecurityScheme` (e.g. Bearer), `OAuth2SecurityScheme`, `OpenIdConnectSecurityScheme`, `MutualTlsSecurityScheme`. HelixAgent's A2A server advertises **HTTPAuthSecurityScheme (Bearer)** to match the existing gateway posture (`API_CONTRACT.md` §3 — `Authorization: Bearer <token>`), so one credential model spans the /v1 gateway and the A2A surface (§3.4).

2. Integration surface — where HelixAgent exposes A2A server + client

2.1 The seam already exists — but the current acp surface is a stub (source-verified)

Two source-verified facts (HelixAgent @17f08ba9, read 2026-07-07):

(a) The acp protocol template — `cmd/api/main.go:124-146` registers a protocol-integration template, and `NewAPIServer (main.go:156-170)` backs the `APIServer` with `services.UnifiedProtocolManager (main.go:162)`:

```
{
    ID:          "acp-agent-communication",
    Name:        "ACP Agent Communication",
    Protocol:    "acp",
    Description: "ACP agent-to-agent communication",
    Protocols:   []string{"acp"},
},
```

(b) A proprietary /api/v1/acp route surface ALREADY EXISTS but its handlers are canned-response STUBS — `cmd/api/main.go:211-215` wires three routes whose handlers (`main.go:370-418`) return static success payloads, NOT real agent-to-agent work:

```
acp := api.Group("/acp")           // → /api/v1/acp
{
    acp.POST("/execute", s.handleACPExecute) // returns
        {"result":"Action executed successfully", ...}
    acp.POST("/broadcast", s.handleACPBroadcast) // returns a
        synthetic broadcast-id; delivers nothing
    acp.GET("/status", s.handleACPStatus) // returns
        hardcoded status:"active" + a HARDCODED capabilities list
}
```

`handleACPExecute (main.go:370-388)` echoes the request with a literal "Action executed successfully"; `handleACPBroadcast (:390-406)` fabricates a `broadcast_id` and reports `delivered_to: len(targets)` without delivering; `handleACPStatus (:408-418)` returns a **hardcoded** capabilities: `["execute_action", "broadcast", "status"]`. This is BLUFF-001/BLUFF-002-class placeholder code (CLAUDE.md §3.3): a print-and-return surface + a hardcoded capability list (a CONST-040 violation the A2A Agent Card MUST NOT reproduce — §2.2 sources `skills[]` from the verifier registry, never a literal).

Honest reconciliation (§11.4.124 investigate-before-wire). The existing `/api/v1/acp/{execute,broadcast,status}` is a **HelixAgent-proprietary ACP shape** (execute/broadcast/status verbs), NOT the Google A2A wire — A2A speaks JSON-RPC-2.0 message `/send/tasks/*` against an Agent Card (§1), a DIFFERENT protocol. P4-T4 therefore does two honest things, neither a silent removal (§11.4.122): (1) it adds the **standards-compliant A2A surface** (§2.2 — Agent Card + JSON-RPC) as the real agent ↔ agent lane, and (2) it treats the three proprietary stubs as **investigate-before-remove candidates** — they are the declared-but-hollow acp wiring point, so the real A2A dispatcher SHOULD back their intent rather than leave a parallel dead surface; whether the proprietary verbs are retired, redirected onto the A2A dispatcher, or kept as an internal convenience API is an operator-gated decision (Q6, §5) because removing a shipped route is §11.4.122-gated. (Catalogue-Check per §11.4.74: extend `HelixDevelopment/helix_agent@17f08ba9` — the A2A surface is a new `internal/a2a` package inside `HelixAgent`, not a duplicate of an existing submodule; the wire types come from `a2aproject/a2a-go` per §1.6.)

2.2 A2A server — HelixAgent is reachable BY other agents

`HelixAgent` exposes an A2A server so any A2A-speaking peer (another `HelixAgent`, a Claude/Gemini/OpenAI agent runtime, a CLI tool) can invoke `HelixAgent`'s capabilities — **including generation routed to the local HelixLLM at :18434** (the coder-fleet serving port, per `03_open_clarifications.md` / `RESUME.md`; the public `HelixLLM` contract is `https://0.0.0.0:8443` per `API_CONTRACT.md` §1).

Load-bearing base-URL gotcha (`RESUME.md` “ENDPOINT GOTCHA”, proven Phase-2 docs/qa/phase2_e2e_20260706/12_endpoint_finding.txt). When `HelixAgent`'s LLM path calls the local coder fleet, the endpoint depends on who appends `/v1`: a raw curl uses `http://localhost:18434/v1/chat/completions`, but an SDK/client that APPENDS `/v1/chat/completions` itself (incl. `HelixAgent` via `HELIX_LLM_LOCAL_OPENAI_ENDPOINT`) MUST be configured with the BASE `http://localhost:18434` (**NO /v1**) — a base of `.../v1` yields a double `/v1/v1/chat/completions` → **HTTP 404**. The A2A message/send handler (below) inherits this: its downstream call reuses `HelixAgent`'s existing config-injected LLM base URL (§11.4.28), so the no-`/v1` base is a configuration fact the design honours, not a new endpoint it invents (§11.4.6).

Registered on the existing gin router (`internal/router/router.go` — same server that already hosts `/health`, `/v1/health`, `/v1/features`, `/metrics`), two new routes, all config-injected (§11.4.28 / `CONST-045/046` — no hardcoded host/port):

Method	Path	Handler	Auth
GET	<code>/.well-known/agent-card.json</code>	serve the Agent Card (public, unauth — discovery)	none (public manifest)
POST	<code>/a2a</code> (config-injectable base, e.g. <code>/v1/a2a</code>)	JSON-RPC 2.0 dispatch (message/send, message/stream, tasks/get, tasks/cancel, tasks/resubscribe, tasks/pushNotificationConfig/*)	Bearer (§1.7 / API_CONTRACT.md §3)

- The **Agent Card** is composed programmatically from the capability surface — `skills[]` sourced from the same registry that feeds `/v1/models + MCP server/discover + CONST-040` verifier capability flags (**NO hardcoded skill list** — CONST-036/040; §11.4.6). One example skill: `generate-code` → advertises that HelixAgent accepts a text Task and returns a code artifact.
- **message/send routing**: the JSON-RPC handler parses the A2A Message, extracts the text/data Parts, and routes to the capability the skill names. For a code/text-generation Task it calls HelixAgent’s existing LLM path which fronts the local HelixLLM (POST `/v1/chat/completions` on the HelixLLM binary, API_CONTRACT.md §4.1), then wraps the real model output as an A2A **Artifact** and transitions the Task submitted → working → completed. **Real HTTP call to the real model — NEVER a simulated response** (BLUFF-001 guard, CLAUDE.md §3.3).
- **message/stream**: reuses the existing SSE writer pattern (HelixLLM’s own streaming path is `text/event-stream` with data: {json} frames, API_CONTRACT.md §4.1 / `streaming.go`); the A2A binding wraps each JSON-RPC Response object in an SSE data: field (Content-Type `text/event-stream`, cited §1.5). No reinvented transport — the QUIC/HTTP3 stack (`internal/transport`) and gin SSE already exist.

2.3 A2A client — HelixAgent reaches OUT to other agents

HelixAgent also ships an A2A client so it can delegate to external A2A agents (the peer-to-peer half). Lives beside the existing MCP host/bridge (`internal/mcp/`) as `internal/a2a/client` (new package, config-driven peer list):

1. GET a peer’s `/.well-known/agent-card.json`, parse `skills + url + security`;
2. select the peer whose skill matches the sub-task;

- 3. message/send (or message/stream) a Task, poll tasks/get (or read the SSE), collect the returned Artifact;
- 4. surface the result back into HelixAgent's own orchestration (internal/agents/registry.go, subagent, swarm).

Peer endpoints + credentials are **config-injected** (.env / config file, §11.4.10 never logged, §11.4.28 decoupled) — never a hardcoded reach into a parent tree (CONST-045).

2.4 Reuse posture (no reinvented transport, §11.4.74)

A2A need	Existing HelixAgent/HelixLLM asset reused
HTTP/TLS listener	gin server + TLS-1.3/HTTP3 stack (API_CONTRACT.md §1; internal/transport/http3.go)
Bearer auth	gateway APIKeyAuth/Bearer middleware (API_CONTRACT.md §3)
SSE streaming	OpenAI SSE writer pattern (API_CONTRACT.md §4.1 / streaming.go)
JSON-RPC + wire types	github.com/a2aproject/a2a-go official SDK (§1.6)
capability/skill source	/v1/models + MCP server/discover + CONST-040 verifier flags
downstream model	local HelixLLM POST /v1/chat/completions (API_CONTRACT.md §4.1)

3. How A2A relates to the already-integrated MCP (the boundary)

A2A and MCP are **orthogonal lanes** and MUST NOT re-implement each other:

Axis	MCP (already integrated)	A2A (this design)
Relationship	agent ↔ tool (host consumes a tool/resource server)	agent ↔ agent (peer delegates a Task to a peer)
HelixAgent role	MCP host consuming CodeGraph + OpenDesign tool servers (internal/mcp/, .mcp.json); AND MCP server exposing Helix tools	A2A server (peers reach Helix capabilities) + A2A client (Helix reaches peers)

Axis	MCP (already integrated)	A2A (this design)
Unit of interaction	a tool call (tools/call) / resource read — synchronous, structured	a Task — first-class, stateful, async, streams status, yields Artifacts
Discovery	MCP server/discover / capability list	Agent Card at /.well-known/agent-card.json
Canonical use	“call CodeGraph’s codegraph_explore” (a TOOL)	“ask another AGENT to complete a coding Task and return the result”

No overlap-reimplementation: the MCP surface (internal/mcp/bridge, CodeGraph/OpenDesign wiring in P5) stays the tool lane; A2A is added as the peer-agent lane. HelixLLM/HelixAgent capabilities are exposed to *tools/editors* via MCP server/discover and to *peer agents* via the A2A Agent Card — the SAME underlying capability registry (CONST-040) feeds both advertisements, so a capability is declared once and surfaced on each channel without duplication. (The operator’s C3 note made this split explicit: editor auto-recognition = the MCP//v1/models lane; A2A = the cross-agent lane.)

4. §11.4.108 runtime signature — the ONE machine-checkable proof

Definition of done for P4-T4: on a **clean deploy** (§11.4.108 / §11.4.139 — HelixAgent’s A2A server freshly booted, pointed at a live local HelixLLM at :18434), the following single machine-checkable signature verifies and is captured to docs/qa/<run-id>/a2a/:

RUNTIME SIGNATURE (A2A end-to-end code generation): a **real A2A client** (the internal/a2a/client, or a curl JSON-RPC driver) performs the full peer flow against the running HelixAgent A2A server:

1. GET /.well-known/agent-card.json → returns a valid Agent Card whose skills[] advertises a generate-code-class skill and whose url + securitySchemes (Bearer) are present;
2. POST /a2a JSON-RPC "message/send" with a real prompt Part (e.g. “Write a Go function that returns the nth Fibonacci number.”) and a valid Bearer token;
3. HelixAgent routes the Task to the **live local HelixLLM** (:18434 → POST /v1/chat/completions) and returns a **real completed Task** — state transitions to "completed" and an **Artifact** carries the real generated Go source;
4. assert: the Task state == "completed", the Artifact text contains **compilable/plausible Go** for the asked function (a content oracle — a func declaration + a return,

not an empty/placeholder body), the returned Task id round-trips through "tasks/get", and the response is a well-formed JSON-RPC 2.0 result.

The captured artefact is the **real JSON-RPC request + response bodies** + the **real generated code** + the PASS/FAIL verdict with its evidence path.

This is a genuine end-to-end proof: it can only PASS if a real A2A client reached a real HelixAgent Agent Card, sent a real Task, HelixAgent made a real HTTP call to a real model, and returned real generated content — impossible to satisfy with a simulated/placeholder handler (BLUFF-001; §11.4 / §107). Feature-class: this adds an agent_interop entry to the §11.4.69 sink-side taxonomy (open to additions), evidence shape = the captured JSON-RPC transcript + generated artifact above.

4.1 Golden-good / golden-bad self-validation (§11.4.107(10))

The A2A acceptance analyzer is **mutation-proofed** with a fixture pair, wired into the meta-test:

- **golden-good fixture** — a captured real transcript where a valid Bearer message/send yields a completed Task with a real code Artifact → analyzer MUST return **PASS**.
- **golden-bad fixtures** (each MUST return **FAIL** — the analyzer cannot be fooled):
 1. **unauthorized Task** — a message/send with a MISSING/invalid Bearer token → the server MUST reject (JSON-RPC error / HTTP 401 per API_CONTRACT.md §3 error envelope), NOT silently accept and process; analyzer FAILs if the transcript shows a processed Task for an unauthenticated caller.
 2. **malformed JSON-RPC** — a body missing "jsonrpc": "2.0" / method / id → the server MUST return a JSON-RPC parse/invalid-request error, never a completed Task.
 3. **placeholder-artifact** — a Task that returns state: "completed" but whose Artifact is empty / a simulated stub string → FAILs the content oracle (no func/return), catching a regressed handler that fakes success.

Paired §1.1 mutation: strip the auth-check OR the content-oracle assertion from the analyzer → a golden-bad fixture (unauthorized Task, or placeholder artifact) PASSes → the gate FAILs. That mutation is the mechanical proof the acceptance test is not itself a bluff (§11.4.120 — the gate + mutation stay a valid pair).

4.2 Four-layer verification (§11.4.108) + honest SKIP boundary

1. **SOURCE** — the `internal/a2a/{server,client}` packages + the Agent Card composer + the two gin routes committed; pre-build grep gate confirms the acp template now resolves to a real A2A handler (no `simulate/placeholder` — BLUFF-001 scan).
2. **ARTIFACT** — the HelixAgent binary builds with the A2A routes registered (`go build ./... exit 0`); the Agent Card endpoint responds on a booted binary.
3. **RUNTIME-ON-CLEAN-TARGET** — the §4 signature verifies against a freshly-booted HelixAgent A2A server + a live local HelixLLM — the definition of done.
4. **USER-VISIBLE** — an external A2A agent / CLI tool discovers HelixAgent's card and delegates a real coding Task, getting real generated code back.

Honest §11.4.3 SKIP boundary (§11.4.6 / §11.4.123). Layer 3 **REQUIRES** a **live local HelixLLM on :18434**, which itself depends on the P0 → P1 GPU foundation (`04_implementation_plan.md` critical path — P0 GPU passthrough → P1 serving core). If that infra is not up when P4-T4 is validated, the runtime signature **MUST** be recorded as **SKIP-with-reason: hardware_not_present** (§11.4.69 closed reason set) + a tracked migration item — **NEVER a metadata-only / “routes registered” PASS** (that is the §11.4 bluff this rule forbids). A **partial** proof **IS** still capturable without a GPU: the Agent-Card discovery (step 1), the auth golden-bad (unauthorized-rejected), and the malformed-JSON-RPC golden-bad can all run against the A2A server with the downstream model stubbed **at the HelixLLM boundary only** (the No-Brain dev fallback, `API_CONTRACT.md` §4.1) — but a **PASS** on the **FULL** end-to-end signature (real generated code) requires the real model and is honestly **SKIP'd** until the serving core exists. The design does not claim the full signature is green today; it specifies exactly what proves it and what blocks it.

5. Open questions (resolve before coding)

- **Q1** Pin `github.com/a2aproject/a2a-go` at a build-verified version targeting Go 1.26 + the v1.0 wire, OR hand-roll the JSON-RPC envelope from the cited method-name strings? (Leaning: use the official SDK per §11.4.74; verify compatibility at build — flagged **UNCONFIRMED** until pinned.)
- **Q2** A2A base path — `/a2a` vs `/v1/a2a` (does it sit inside the existing `/v1` Bearer group, or its own group)? Config-injected either way (§11.4.28).
- **Q3** Which capabilities does the Agent Card advertise as `skills[]` at launch — only `generate-code`, or the full **CONST-040** capability set (`generate`,

embed, translate, vision...)? (Sourced from the verifier registry, never hardcoded.)

- **Q4** Push-notification (webhook) config support (`tasks/pushNotificationConfig/*`) — ship in P4-T4 or defer? (Async long-task callbacks are optional per the Agent Card capabilities.pushNotifications flag.)
 - **Q5** gRPC / HTTP+JSON bindings — documented future extension; confirm JSON-RPC-only is acceptable for the first release.
 - **Q6** (operator-gated, §11.4.122) Fate of the existing proprietary `/api/v1/acp/{execute,broadcast,status}` stubs (§2.1b): retire them, redirect them onto the real A2A dispatcher, or keep them as an internal convenience API? Removing a shipped route is §11.4.122-gated — the operator decides; the design does NOT silently drop them.
-

6. Composition footer — constitutional anchors touched

- **§11.4.6** (no-guessing) — protocol facts cited to the LATEST fetched spec; every latency figure (`EST - measure`); the v1.0-proto-vs-JSON-RPC-binding tension flagged (§0.1), not hidden; the missing-GPU SKIP boundary stated (§4.2).
- **§11.4.66 / §11.4.105** (operator decision honoured) — C3 “Google A2A” adopted verbatim; the protocol is NOT re-decided.
- **§11.4.74** (extend-don’t-reimplement) — reuse the existing gin/TLS/SSE stack + the official `a2aproject/a2a-go` SDK + the existing `acp` protocol template; no reinvented transport.
- **§11.4.124** (investigate-before-wire) — the `acp-agent-communication` template is the intended wiring point, filled in rather than duplicated.
- **§11.4.28 / CONST-045 / CONST-046** (decoupled, config-injected) — peer endpoints, ports, base path, credentials all config-injected; never hardcoded.
- **CONST-036 / CONST-040** (verifier single source of truth) — Agent Card `skills[]` + capability flags sourced from the verifier registry, never a hardcoded list.
- **§11.4.99 / §11.4.150** (latest-source + deep multi-angle) — A2A facts from the LATEST official spec (`a2a-protocol.org` latest + v0.3.0) + the GitHub project (governance + Go SDK), ≥ 2 distinct angles, cited with access date.
- **§11.4.107(10)** (self-validated analyzer) — golden-good + three golden-bad fixtures (unauthorized, malformed, placeholder-artifact).
- **§11.4.108 / §11.4.139** (four-layer runtime-signature on a clean target) — the §4 A2A end-to-end code-generation signature is the definition of done.
- **§11.4.69** (sink-side evidence taxonomy) — adds an `agent_interop` feature class; evidence = captured JSON-RPC transcript + real generated artifact.

- §11.4.3 / §11.4.123 (honest SKIP, never a metadata PASS) — full signature SKIP'd hardware_not_present until the P1 serving core is live; partial proofs enumerated.
- **BLUFF-001** (CLAUDE.md §3.3) — message/send makes a REAL model call; never a simulated response.

Sources verified

Deep-research 2026-07-07: - <https://a2a-protocol.org/latest/specification/> - <https://a2a-protocol.org/v0.3.0/specification/> - <https://github.com/a2aproject/A2A>

(Negative finding, §11.4.99(B), all three URLs re-fetched + independently verified 2026-07-07: the /latest/specification/ page self-identifies its released version as 1.0.0, is **proto-first** (spec/a2a.proto authoritative), and exposes ONLY the proto-style names — the TaskState enum (TASK_STATE_SUBMITTED ... TASK_STATE_UNSPECIFIED), Message.role (ROLE_USER/ROLE_AGENT), the Part oneof (text/raw/url/data), the three transport bindings (§§9–11), and the five *SecurityScheme types. It does NOT carry the well-known-URI path, the JSON-RPC method-name STRINGS, the TaskState JSON wire values, or the SSE detail — so those four were verified verbatim on the /v0.3.0/specification/ JSON-RPC-binding page, which states the Agent Card location `https:// {server_domain}/.well-known/agent-card.json`, enumerates "message/send"/"message/stream"/"tasks/get"/"tasks/cancel"/"tasks/resubscribe"/"tasks/pushNotificationConfig/{set,get,list,delete}"/"agent/getAuthenticatedExtendedCard", the TaskState JSON values incl. the "unknown" catch-all, and streaming as Content-Type: text/event-stream where each SSE data field is a complete JSON-RPC 2.0 Response. The GitHub project page confirms verbatim "*an open source project under the Linux Foundation, contributed by Google*", latest release **v1.0.1 (2026-05-28)**, and the Go SDK go get github.com/a2aproject/a2a-go. Internal HelixAgent facts are source-verified against submodules/helix_agent@17f08ba9: the acp-agent-communication template (cmd/api/main.go:140-144), the UnifiedProtocolManager (main.go:162), and — corrected in §2.1 vs the first draft — the **pre-existing proprietary /api/v1/acp/{execute,broadcast,status} routes** (main.go:211-215) whose handlers are canned-response STUBS (main.go:370-418, incl. a hardcoded capability list at :416). The HelixLLM TLS/auth/route posture + the no-/v1 base-URL gotcha are source-verified against submodules/helix_llm/docs/API_CONTRACT.md §1/§3/ §4.1 + RESUME.md.)